

```
template <simd_integral V>
V f(V x) {
    return x + 1; // ill-formed for V::value_type = (u)int8_t, (un)int16_t, and uint32_t
}
```

needs to use `x + V(1)`³. A clever user might write `x + '1'` instead. But that fails for the `char` type with different signedness.

Consequently, users would need to get used to writing explicit conversions for the constants they use in `std::simd` code. That's not only verbose and ugly, it is also error-prone. Whenever we coerce our users into writing explicit conversions, then value-changing conversions cannot be diagnosed as erroneous anymore. An explicit `static_cast<uint64_t>(-1)` means `0xffff'ffff'ffff'ffff`, whereas `uint64_t x = -1` could have been intended to mean `0x0000'0000'ffff'ffff` or is a result of a logic flaw in the code. E.g., GCC's `-Wsign-conversion` diagnoses the latter, but not the former⁴.

If, with C++26, our users are starting to explicitly convert their `int` constants to `basic_vec`, then the interface of `basic_vec` is at least in part guilty for introducing harder to find bugs.

Tony Table 2 presents an example of the solution⁵. Note that the code on the left will never warn about the value-changing conversion, even with all conversion related warnings enabled. This is due to the explicit conversion, which is telling the compiler “I know what I'm doing; no need to warn me about it”.

before	with P4012R0
<pre>template <simd_floating_point V> V f(V x) { return x + V(0x5EAF00D); } f(vec<double>()); // OK // compiles but adds 99282960 instead of 99282957 f(vec<float>()); // compiles but adds infinity instead of 99282957 f(vec<std::float16_t>());</pre>	<pre>template <simd_floating_point V> V f(V x) { return x + 0x5EAF00D; } f(vec<double>()); // OK // ill-formed: value-changing conversion f(vec<float>()); // ill-formed: value-changing conversion f(vec<std::float16_t>());</pre>

Before/After Table 2: Add an offset

A safer implementation of the code on the left side of Tony Table 2 (without this paper) would have been to write `x + std::cw<0x5EAF00D>` instead. Then the value-changing conversion would have resulted in a constraint failure on the broadcast constructor. However, `V(0x5EAF00D)` is shorter and needs fewer template instantiations. I expect most users (including myself) will/do not use `std::cw` all over the place.

6

DESIGN SPACE

In the design review of P1928 of this issue of the broadcast constructor it was overlooked (and never discussed) that a `constexpr` overload of the broadcast constructor could solve this problem. Before

3 explicit conversion to `basic_vec` allows conversions that are not value-preserving
4 And that's useful, because the former says “I'm intentionally doing this conversion, no need to warn.”
5 I got bitten by this in my `std::simd` unit tests